School of Future Tech

Case Study Report

Snake Game Deluxe Web Edition (Arcade Gaming Platform)

by

AASHIKA PANDEY
150096725010

B.Tech CSE (2025–29)
ITM Skills University

---

1. Introduction to the Case Study

The Snake Game Deluxe Web Edition is a browser-based arcade game developed using HTML5, CSS, and JavaScript. The project recreates the classic snake game and enhances it with additional modern features such as score tracking, leaderboard functionality, obstacles, and a forum section for user interaction.

The traditional snake game is a simple game where the player controls a snake that moves around a grid. The objective of the game is to eat food items that appear randomly on the screen. Every time the snake eats food, its length increases and the player's score increases.

In this project, the classic snake game concept is improved by introducing additional gameplay elements. The game includes increasing speed levels, obstacles that make gameplay more challenging, and a leaderboard where players can compare scores.

The entire application is developed using frontend technologies and runs directly in a web browser without requiring any backend server. Player scores and comments are stored using the browser's localStorage feature.

This case study focuses on designing, developing, and implementing an interactive web-based arcade game using modern web development techniques.

Caption example:**Figure 1: Snake Game Interface**

---

2. Problem Statement

Classic snake games available online are often very basic and lack modern user interface design and interactive features. Most versions do not include score tracking systems or any way for players to compare their performance with others.

In addition, many snake games do not include community interaction features such as forums or comment sections where users can share gameplay strategies and tips.

Because of these limitations, players have limited engagement and cannot interact with other players.

The objective of this project is to develop a **responsive Snake Game web application** that includes:

• Real-time score tracking
• Leaderboard system to compare scores
• Forum section for user interaction
• Increasing difficulty levels during gameplay
• Obstacle mode for additional challenge

The goal is to create a more engaging and modern version of the classic snake game.

3. Case Study Design

The Snake Game Deluxe Web Edition is designed using three main frontend technologies.

HTML (HyperText Markup Language)

HTML is used to create the basic structure of the webpage. It defines the layout of the game including the game canvas, navigation menu, leaderboard section, and forum comment area.

The canvas element is used to create the game board where the snake and food are displayed.

index.html > html

```html
<!DOCTYPE html>
<html>
<head>
<title>Snake Game Deluxe</title>
<link rel="stylesheet" href="style.css">
</head>

<body>

<h1>Snake Game Deluxe</h1>

<div class="nav">
<button onclick="showSection('game')">Play</button>
<button onclick="showSection('leaderboard')">Leaderboard</button>
<button onclick="showSection('forum')">Forum</button>
</div>

<!-- GAME SECTION -->
<div id="game" class="section">

<h2>Play Game</h2>

<p>Score: <span id="score">0</span></p>

<canvas id="gameCanvas" width="400" height="400"></canvas>

<br>

<button onclick="startGame()">Start Game</button>
<button onclick="pauseGame()">Pause</button>
<button onclick="resumeGame()">Resume</button>

<!-- GAME OVER SCREEN -->
<div id="gameOverBox" class="gameover">

<h3>Game Over</h3>

<p>Your Score: <span id="finalScore"></span></p>

<input type="text" id="playerName" placeholder="Enter your name">

<br><br>

<button onclick="saveScore()">Save Score</button>
<button onclick="startGame()">Play Again</button>

</div>

</div>

<!-- LEADERBOARD -->
<div id="leaderboard" class="section" style="display:none">

<h2>Leaderboard</h2>

<ul id="scoreList"></ul>

</div>

<!-- FORUM -->
<div id="forum" class="section" style="display:none">

<h2>Forum</h2>

<input type="text" id="commentInput" placeholder="Write a comment">

<button onclick="addComment()">Post</button>

<ul id="commentList"></ul>

</div>

<script src="script.js"></script>

</body>
</html>
```

Figure 2: HTML Structure of the Game

CSS (Cascading Style Sheets)

CSS is used to design and style the user interface of the application. It controls the layout, colors, button design, and overall visual appearance of the game.

The goal of the design is to keep the interface simple, clean, and easy to use so that players can focus on the gameplay.

style.css > .gameover

```css
body{
font-family: Arial, Helvetica, sans-serif;
background: #f4f4f4;
text-align:center;
margin:0;
}

h1{
background: #333;
color: white;
padding:15px;
margin:0;
}

.nav{
margin:20px;
}

button{
padding:10px 20px;
margin:5px;
border:none;
background: #333;
color: white;
cursor:pointer;
}

button:hover{
background: #555;
}

canvas{
background: white;
border:2px solid #333;
margin-top:10px;
}

.section{
margin-top:20px;
}

input{
padding:8px;
}

ul{
list-style:none;
padding:0;
}

li{
background: white;
margin:5px auto;
padding:8px;
width:250px;
border:1px solid #ccc;
}

/* GAME OVER BOX */

.gameover{
display:none;
background: white;
border:2px solid #333;
padding:20px;
width:250px;
margin:20px auto;
}
```

Figure 3: CSS Styling for Game Interface

JavaScript

JavaScript is used to implement the main functionality of the game. It controls snake movement, food generation, collision detection, score calculation, and game speed.

JavaScript also stores player scores and forum comments using localStorage.

JS script.js > showSection

```javascript
let canvas = document.getElementById("gameCanvas");
let ctx = canvas.getContext("2d");
let snake;
let food;
let dx;
let dy;
let score;
let gameInterval;
let gameSpeed = 120;
let paused = false;
let obstacles = [
{x:100,y:200},
{x:300,y:100},
{x:200,y:300}
];
document.addEventListener("keydown", changeDirection);
function startGame(){
snake = [{x:200,y:200}];
dx = 20;
dy = 0;
score = 0;
gameSpeed = 120;
paused = false;
food = {
x: Math.floor(Math.random()*20)*20,
y: Math.floor(Math.random()*20)*20
};
document.getElementById("score").innerText = score;
document.getElementById("gameOverBox").style.display="none";
clearInterval(gameInterval);
gameInterval = setInterval(gameLoop,gameSpeed);
}
function gameLoop(){
if(paused) return;
moveSnake();
checkCollision();
drawGame();
increaseSpeed();
}
function moveSnake()
{
let head = {x:snake[0].x + dx, y:snake[0].y + dy};
snake.unshift(head);
if(head.x == food.x && head.y == food.y){
score += 10;
document.getElementById("score").innerText = score;
food = {
x: Math.floor(Math.random()*20)*20,
y: Math.floor(Math.random()*20)*20
};
}else{
snake.pop();
}
}
function drawGame()
{
ctx.clearRect(0,0,400,400);
ctx.fillStyle="red";
ctx.fillRect(food.x,food.y,20,20);
ctx.fillStyle="green";
snake.forEach(part=>{
ctx.fillRect(part.x,part.y,20,20);
});
ctx.fillStyle="gray";
obstacles.forEach(o=>{
ctx.fillRect(o.x,o.y,20,20);
});
}
function changeDirection(e){
if(e.key=="ArrowUp" && dy==0){dx=0;dy=-20;}
if(e.key=="ArrowDown" && dy==0){dx=0;dy=20;}
if(e.key=="ArrowLeft" && dx==0){dx=-20;dy=0;}
if(e.key=="ArrowRight" && dx==0){dx=20;dy=0;}
}
function checkCollision(){
let head = snake[0];
if(head.x<0 || head.x>=400 || head.y<0 || head.y>=400){
gameOver();
}
for(let i=1;i<snake.length;i++){
```

```
if(head.x==snake[i].x && head.y==snake[i].y){
gameOver();
}
}
obstacles.forEach(o=>{
if(head.x==o.x && head.y==o.y){
gameOver();
}
});
}
function increaseSpeed(){
if(score % 50 == 0 && score !=0){
clearInterval(gameInterval);
gameSpeed -= 10;
if(gameSpeed < 60){
gameSpeed = 60;
}
gameInterval = setInterval(gameLoop,gameSpeed);
}
}
function gameOver(){
clearInterval(gameInterval);
document.getElementById("finalScore").innerText = score;
document.getElementById("gameOverBox").style.display="block";
}
function saveScore(){
let name = document.getElementById("playerName").value;
if(name==="") name="Player";
let scores = JSON.parse(localStorage.getItem("scores")) || [];
scores.push({name:name,score:score});
scores.sort((a,b)=>b.score-a.score);
localStorage.setItem("scores",JSON.stringify(scores));
document.getElementById("playerName").value="";
displayScores();
}
function pauseGame(){
paused = true;
}
function resumeGame(){
paused = false;
}
function displayScores(){
let list = document.getElementById("scoreList");
list.innerHTML="";
let scores = JSON.parse(localStorage.getItem("scores")) || [];
scores.forEach(s=>{
let li = document.createElement("li");
li.innerText = s.name + " - " + s.score;
list.appendChild(li);
});
}
function addComment(){
let input = document.getElementById("commentInput");
let comments = JSON.parse(localStorage.getItem("comments")) || [];
comments.push(input.value);
localStorage.setItem("comments",JSON.stringify(comments));
input.value="";
displayComments();
}
function displayComments(){
let list = document.getElementById("commentList");
list.innerHTML="";
let comments = JSON.parse(localStorage.getItem("comments")) || [];
comments.forEach(c=>{
let li = document.createElement("li");
li.innerText = c;
list.appendChild(li);
});
}
function showSection(id){
document.querySelectorAll(".section").forEach(sec=>{
sec.style.display="none";
});
document.getElementById(id).style.display="block";
if(id=="leaderboard") displayScores();
```

Figure 4: JavaScript Game Logic

---

4. Methods & Algorithms Used

Several basic algorithms and programming concepts are used to implement the functionality of the snake game.

Snake Movement Algorithm

The snake moves continuously within the canvas using keyboard arrow key inputs. The position of the snake's head is updated every few milliseconds, and the rest of the body follows the head.

Food Generation

Food appears randomly on the game board. When the snake reaches the food position, the snake grows longer and the player's score increases.

Collision Detection

The system checks for three types of collisions:

• Collision with the wall
• Collision with the snake's own body
• Collision with obstacles placed in the game

If any collision occurs, the game ends and the game over screen is displayed.

Speed Increase

As the player score increases, the speed of the snake increases. This makes the game progressively more challenging.

---

5. Implementation

The game is implemented using HTML Canvas to draw the snake, food, and obstacles.

The snake is represented as a series of small blocks that move together. Each time the snake eats food, a new block is added to the snake body.

The game also includes additional features such as:

• Obstacle mode
• Pause and resume functionality
• Game over screen displaying the final score
• Leaderboard system for storing player scores
• Forum section where players can share comments

The leaderboard and forum comments are stored using the browser's localStorage feature.

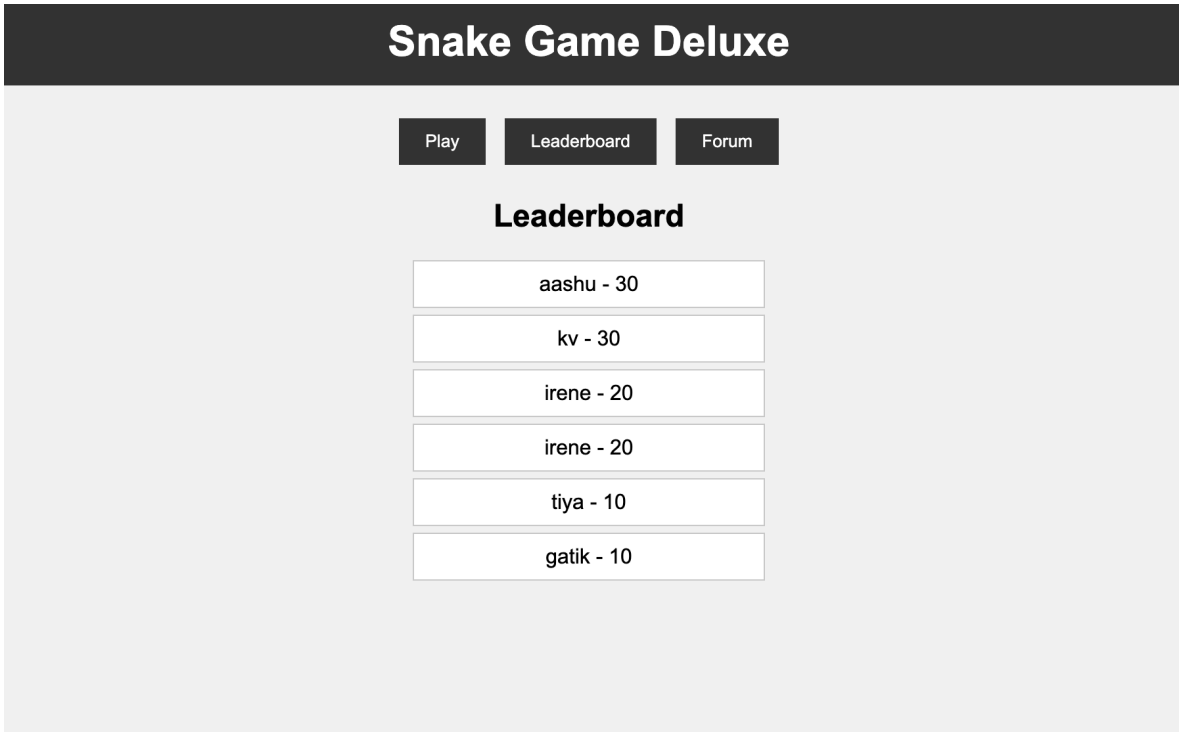

Figure 5: Leaderboard and Forum Section

---

6. Results and Conclusion

The Snake Game Deluxe Web Edition successfully demonstrates the development of an interactive browser-based game using HTML, CSS, and JavaScript.

The project improves the traditional snake game by introducing modern gameplay features such as speed increase, obstacle mode, leaderboard tracking, and a forum section for user interaction.

Through this project, important web development concepts such as HTML canvas, JavaScript event handling, and localStorage data storage are implemented.

This project provides practical experience in developing interactive web applications and demonstrates how frontend technologies can be used to build engaging browser-based games.

---

7. References

MDN Web Docs – HTML Canvas API
https://developer.mozilla.org

W3Schools – HTML, CSS and JavaScript Tutorials
https://www.w3schools.com

JavaScript Game Development Guide
https://developer.mozilla.org/en-US/docs/Games

---

8. GitHub link

snake game